

Implementation Guide — Clean Architecture Migration

Ce guide explique comment tester et utiliser les nouveaux DataSources réels dans AutoSuivi.tn.

Architecture résultante

```

src/
├── core/
│   ├── network/
│   │   ├── ApiClient.ts        ← Client Axios centralisé (JWT, intercepteurs)
│   │   └── index.ts
│   └── domain/
│       ├── entities/          ← Entités métier (inchangées)
│       │   ├── Vehicle.ts
│       │   ├── MileageReading.ts
│       │   ├── User.ts
│       │   ├── Maintenance.ts
│       │   └── Deadline.ts
│       └── repositories/      ← NEW Interfaces (contrats domain)
│           ├── IVehicleRepository.ts
│           ├── IMileageRepository.ts
│           ├── IAuthRepository.ts
│           └── index.ts
├── data/
│   ├── datasources/
│   │   ├── MockDataSource.ts   ← Conservé pour maintenance/deadlines
│   │   └── remote/            ← NEW Appels API réels
│   │       ├── VehicleRemoteDataSource.ts
│   │       ├── MileageRemoteDataSource.ts
│   │       ├── AuthRemoteDataSource.ts
│   │       └── index.ts
│   ├── repositories/         ← NEW Implémentations concrètes
│   │   ├── VehicleRepositoryImpl.ts
│   │   ├── MileageRepositoryImpl.ts
│   │   └── index.ts
│   ├── services/
│   │   └── KmCalculator.ts      ← Inchangé
├── presentation/
│   ├── context/
│   │   ├── AuthContext.tsx    ← ♻️ Refactorisé (API au lieu de mock)
│   │   └── VehicleContext.tsx ← ♻️ Refactorisé (repositories au lieu
└── d'AsyncStorage)
  
```

Fichiers créés

Fichier	Rôle
<code>src/core/network/ApiClient.ts</code>	Client HTTP Axios avec intercepteurs JWT, conversion snake_case↔camelCase, gestion 401
<code>src/domain/repositories/IVehicleRepository.ts</code>	Interface du repository véhicules
<code>src/domain/repositories/IMileageRepository.ts</code>	Interface du repository kilométrage
<code>src/domain/repositories/IAuthRepository.ts</code>	Interface du repository authentification
<code>src/data/datasources/remote/VehicleRemoteDataSource.ts</code>	Appels HTTP pour CRUD véhicules
<code>src/data/datasources/remote/MileageRemoteDataSource.ts</code>	Appels HTTP pour lectures kilométriques
<code>src/data/datasources/remote/AuthRemoteDataSource.ts</code>	Appels HTTP pour login/register/profile
<code>src/data/repositories/VehicleRepositoryImpl.ts</code>	Implémentation concrète IVehicleRepository
<code>src/data/repositories/MileageRepositoryImpl.ts</code>	Implémentation concrète IMileageRepository

Fichiers modifiés

Fichier	Changements
<code>src/presentation/context/AuthContext.tsx</code>	Login/signup via API réelle, gestion JWT, détection 401
<code>src/presentation/context/VehicleContext.tsx</code>	CRUD véhicules via API, mileage via API, maintenance/deadlines encore en local

Comment tester

Prérequis

1. Le backend doit être accessible à `http://102.204.205.49/api`
2. Les endpoints auth (`/auth/login` , `/auth/register`) doivent être fonctionnels

3. Les endpoints véhicules (/vehicles) doivent être fonctionnels

Test de l'authentification

```
# Vérifier que le login fonctionne
curl -X POST http://102.204.205.49/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email": "test@test.com", "password": "password123"}'

# Réponse attendue : { "token": "eyJ...", "user": { ... } }
```

Test des véhicules

```
# Lister les véhicules (avec le token obtenu)
TOKEN="votre_token_jwt"
curl -X GET http://102.204.205.49/api/vehicles \
  -H "Authorization: Bearer $TOKEN"

# Créer un véhicule
curl -X POST http://102.204.205.49/api/vehicles \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"brand": "Peugeot", "model": "208", "year": 2020, "plate": "156 TU 7890",
    "initial_mileage": 15000, "current_mileage": 15000}'
```

Test du kilométrage

```
# Ajouter une lecture
curl -X POST http://102.204.205.49/api/vehicles/VEHICLE_ID/mileage \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"value": 16500, "date": "2025-05-08T10:00:00Z"}'
```

Test dans l'app

1. Lancer l'app : `npx expo start`
2. Se connecter avec des identifiants valides
3. Vérifier que la liste des véhicules se charge
4. Ajouter un véhicule → vérifier qu'il apparaît
5. Ajouter une lecture kilométrique → vérifier la mise à jour

Gestion des erreurs

L'ApiClient gère automatiquement :

- **401 Unauthorized** → Déconnexion automatique + redirection vers login
- **Erreurs réseau** → Message d'erreur propagé via `ApiError`
- **Timeout** → Après 15 secondes

Le VehicleContext expose un champ `error` pour afficher les erreurs dans l'UI.

Migration progressive

La migration est **incrémentale** :

Fonctionnalité	Source de données	État
Authentification	✓ API Backend	Migré
Véhicules (CRUD)	✓ API Backend	Migré
Kilométrage	✓ API Backend	Migré
Maintenance	⌚ AsyncStorage (local)	À migrer quand le backend sera prêt
Échéances	⌚ AsyncStorage (local)	À migrer quand le backend sera prêt
Intervalles entretien	📱 AsyncStorage (settings)	Reste local (paramètre utilisateur)

Prochaines étapes

1. **Créer les endpoints maintenance** sur le backend et migrer `MaintenanceRemoteDataSource`
2. **Créer les endpoints deadlines** sur le backend et migrer `DeadlineRemoteDataSource`
3. **Ajouter un cache offline** avec AsyncStorage comme couche de fallback
4. **Implémenter le refresh token** si le backend le supporte
5. **Ajouter des tests unitaires** pour les DataSources et Repositories